

Generics

Swift Programming

[Swift Documentation : Generics](#)

Introduction

- Swift Generics allows us to create a single function and class (or any other types) that can be used with different data types.
- This helps us to reuse our code.

Swift Generic Function

- In Swift, we can create a function that can be used with any type of data. Such a function is known as a Generic Function.
- Here's how we can create a generic function in Swift:

```
// create a generic function
func displayData<T>(data: T){
    ...
}
```

Here,

- We have created a generic function named `displayData()`.
- T used inside the angle bracket `<>` is called the **type parameter**.
- And based on the type of the value passed to the function, the T is replaced by that data type (Int, String, and so on).
- This placeholder tells Swift that we will be **defining the type at runtime**. We can then use that placeholder type in place of any type definition within the parameter definitions, the return type, or the function itself.

Example

```
// create a generic function
func displayData<T>(data: T) {
    print("Generic Function:")
    print("Data Passed:", data)
}

// generic function working with String
displayData(data: "Swift")

// generic function working with Int
displayData(data: 5)
```

OUTPUT:

```
Generic Function:
Data Passed: Swift
Generic Function:
Data Passed: 5
```


Swift Generic Class

- Similar to the generic function, we can also create a class that can be used with any type of data. Such a class is known as **Generic Class**.

```
// create a generic class
class Information<T> {

    // property of T type
    var data: T

    init (data: T) {
        self.data = data
    }

    // method that return T type variable
    func getData() -> T {
        return self.data
    }
}
```

Output:

```
Generic Class returns: 6
Generic Class returns: Swift
```

```
// initialize generic class with Int data
var intObj = Information<Int>(data: 6)
print("Generic Class returns:", intObj.getData())

// initialize generic class with String data
var strObj = Information<String>(data: "Swift")
print("Generic Class returns:", strObj.getData())
```

Type Constraints in Swift Generics

- In general, the **type parameter** can accept any data type (Int, String, Double, ...).
- However, if we want to use generics for some specific types (such as accepting data of number types) only, then we can use type constraints.
- Here's how we create type constraints:

```
func addition<T: Numeric>(num1: T, num2: T) {  
    ...  
}
```

- Here, <T: Numeric> adds constraints to the type parameter. It defines that T needs to conform to the Numeric protocol.
- **Note:** Numeric is the built-in protocol for numeric values like Int and Double.

Example: Type Constraints

```
//create a generic function with type constraint
func addition<T: Numeric>(num1: T, num2: T) {

    print("Sum:", num1 + num2)
}

// pass Int value
addition(num1: 5, num2: 10)

// pass Double value
addition(num1: 5.5, num2: 10.8)
```

OUTPUT:

Sum: 15

Sum: 16.3

Naming Type Parameters

- In most cases, type parameters have descriptive names, such as `Key` and `Value` in `Dictionary<Key, Value>` and `Element` in `Array<Element>`, which tells the reader about the relationship between the type parameter and the generic type or function it's used in.
- However, when there isn't a meaningful relationship between them, it's traditional to name them using single letters such as `T`, `U`, and `V`, such as `T` in the `swapTwoValues(_:_:)` function.
- Use upper camel case names for type parameters, like `T` and `MyTypeParameter`, to indicate that they're a placeholder for a *type*, not a value.

Multiple Type Parameters on Swift Generics

- Multiple generic types: we can create multiple placeholders by separating them with commas.

```
func testGeneric(a:T, b:E) {  
    print("\(a) \ (b)")  
}
```

- Here, we are defining two generic placeholders, **T** and **E**. In this case, we can set the **T** placeholder to one type and the **E** placeholder to a different type.

Custom Generic Types

- In addition to generic functions, Swift enables you to define your own *generic types*.
- These are custom classes, structures, and enumerations that can work with *any* type, in a similar way to `Array` and `Dictionary`.
- This section shows you how to write a generic collection type called `Stack`.
- A stack is an ordered set of values, similar to an array, but with a more restricted set of operations than Swift's `Array` type.
- An array allows new items to be inserted and removed at any location in the array. A stack, however, allows new items to be appended only to the end of the collection (known as ***pushing*** a new value on to the stack).
- Similarly, a stack allows items to be removed only from the end of the collection (known as ***popping*** a value off the stack).

Note:-

- The concept of a stack is used by the `UINavigationController` class to model the view controllers in its navigation hierarchy.
- You call the `UINavigationController` class `pushViewController(_:animated:)` method to add (or push) a view controller on to the navigation stack, and its `popViewControllerAnimated(_:)` method to remove (or pop) a view controller from the navigation stack.
- A stack is a useful collection model whenever you need a strict “last in, first out” approach to managing a collection.

Custom Generic Types

```
struct Stack<Element> {  
    var items: [Element] = []  
    mutating func push(_ item: Element) {  
        items.append(item)  
    }  
    mutating func pop() -> Element {  
        return items.removeLast()  
    }  
}
```

Custom Generic Types

- `Element` defines a placeholder name for a type to be provided later. This future type can be referred to as **`Element`** anywhere within the structure's definition.
- In this case, `Element` is used as a placeholder in three places:
 - To create a property called `items`, which is initialized with an empty array of values of type `Element`
 - To specify that the `push(_ : )` method has a single parameter called `item`, which must be of type `Element`
 - To specify that the value returned by the `pop( )` method will be a value of type `Element`
- Because it's a generic type, `Stack` can be used to create a stack of *any* valid type in Swift, in a similar manner to `Array` and `Dictionary`.

Custom Generic Types

- You create a new Stack instance by writing the type to be stored in the stack within angle brackets. For example, to create a new stack of strings, you write `Stack<String>()`:

```
var stackOfStrings = Stack<String>()  
stackOfStrings.push("one")  
stackOfStrings.push("two")  
stackOfStrings.push("three")  
stackOfStrings.push("four")  
// the stack now contains 4 strings
```

- Popping a value from the stack removes and returns the top value, "four":

```
let fromTheTop = stackOfStrings.pop()  
// fromTheTop is equal to "four", and the stack now contains 3 strings
```

Extending a Generic Type

- When you extend a generic type, you don't provide a type parameter list as part of the extension's definition.
- Instead, the type parameter list from the *original* type definition is available within the body of the extension, and the original type parameter names are used to refer to the type parameters from the original definition.
- The following example extends the generic Stack type to add a read-only computed property called `topItem`, which returns the top item on the stack without popping it from the stack:

```
extension Stack {  
    var topItem: Element? {  
        return items.isEmpty ? nil : items[items.count - 1]  
    }  
}
```

- The `topItem` property returns an optional value of type `Element`. If the stack is empty, `topItem` returns `nil`; if the stack isn't empty, `topItem` returns the final item in the `items` array.
- The `topItem` computed property can now be used with any `Stack` instance to access and query its top item without removing it.

```
if let topItem = stackOfStrings.topItem {  
    print("The top item on the stack is \(topItem).")  
}  
// Prints "The top item on the stack is tres."
```


Example:

```
1. struct Stack<Element> {  
    var items: [Element] = []  
  
    mutating func push(_ item: Element) {  
        items.append(item)  
    }  
  
    mutating func pop() -> Element {  
        return items.removeLast()  
    }  
}
```

```
2. extension Stack {  
    // A read-only computed property  
    var topItem: Element? {  
        // Returns nil if empty, otherwise returns the last item without popping  
        return items.isEmpty ? nil : items[items.count - 1]  
    }  
}
```

```
3. // Create a stack of Strings  
var stackOfStrings = Stack<String>()  
stackOfStrings.push("uno")  
stackOfStrings.push("dos")  
stackOfStrings.push("tres")  
  
// Use the new property from the extension  
if let topItem = stackOfStrings.topItem {  
    print("The top item on the stack is \(topItem).")  
    // Output: The top item on the stack is tres.  
}  
  
// Verification: The item is still there because we didn't 'pop' it  
print("Current count: \(stackOfStrings.items.count)")  
// Output: Current count: 3
```

Using Generics to Simplify Array Functions

- Arrays are fundamental data structures used to store collections of elements. Functions that operate on arrays (like sorting, filtering, mapping, etc.) can often be made more concise and reusable using generics.
- **Without Generics:** You'd have to write separate functions for arrays of integers, doubles, strings, etc.
- **With Generics:** You can write a single function that works for any array where the elements are comparable (i.e., you can use the < or > operators).

Using Generics to Simplify Array Functions : Example

// Generic function to find the maximum element in an array

```
func findMax<T: Comparable>(_ array: [T]) -> T? {  
    guard !array.isEmpty else {  
        return nil  
    } // Handle empty array case  
  
    var maxElement = array[0] //Initialize with the first element  
    for element in array {  
        if element > maxElement {  
            maxElement = element  
        }  
    }  
    return maxElement  
}
```


Using Generics to Simplify Array Functions : Example

```
// Generic function to find the maximum element in an array
func findMax<T: Comparable>(_ array: [T]) -> T? {
    guard !array.isEmpty else {
        return nil
    }
    // Handle empty array case

    var maxElement = array[0] //Initialize with the
    first element
    for element in array {
        if element > maxElement {
            maxElement = element
        }
    }
    return maxElement
}
```

/ Example usage with different array types:

```
let intArray = [3, 1, 4, 1, 5, 9, 2, 6]
if let maxInt = findMax(intArray) {
    print("Max integer: \(maxInt)")
    // Output: Max integer: 9
}
```

```
let doubleArray = [1.5, 2.7, 0.8, 3.2]
if let maxDouble = findMax(doubleArray) {
    print("Max double: \(maxDouble)")
    // Output: Max double: 3.2
}
```

```
let stringArray = ["apple", "banana", "cherry", "date"]
if let maxString = findMax(stringArray) {
    print("Max string (lexicographically): \(maxString)")
    // Output: Max string (lexicographically): date
}
```

Advanced Generic Type Constraints

1. Advanced Type Constraints

- Type constraints go beyond simple protocol conformance. They allow you to add more specific requirements on the types used with generics.
 - **Class Constraints:** You can constrain a generic type to be a specific class or a subclass of it.
 - **Associated Type Constraints (in Protocols):** You can add constraints to associated types within protocols, making them more flexible.
 - **Where Clauses:** These allow you to add more complex constraints using `where` clauses.

Advanced Generic Type Constraints

Associated types

- An associated type declares a placeholder name that can be used instead of a type within a protocol.
- The actual type to be used is not specified until the protocol itself is adopted.
- When creating generic functions and types, we used a very similar syntax.
- Defining associated types for a protocol, however, is a little different. We specify an associated type using the `associatedtype` keyword.
- For this example, we will create a simple protocol named `MyProtocol`:

```
protocol MyProtocol {  
    associatedtype E  
    var items: [E] {get set}  
    mutating func add(item: E)  
}
```


Advanced Generic Type Constraints

Associated types

- For this example, we will create a simple protocol named **MyProtocol**:

```
protocol MyProtocol {  
    associatedtype E  
    var items: [E] {get set}  
    mutating func add(item: E)  
}
```

- In this protocol, we declare an associated type named **E**. We use associated type as the type for the **items** array, as well as the parameter type for the **add(item:)** method.
- Now create types that conform to this protocol by providing either a concrete type or a generic type for the associated type.

```
struct MyIntType: MyProtocol {  
    var items: [Int] = []  
    mutating func add(item: Int) {  
        items.append(item)  
    }  
}
```

- Here, we create a type named **MyIntType** that conforms to the **MyProtocol** protocol. We then implement the **items** array and the **add(item:)** method using the integer type.

Advanced Generic Type Constraints

Associated types

- let's see how we would use a generic type when creating a type that conforms to the `MyProtocol` protocol:

```
struct MyGenericType<T>: MyProtocol {  
    var items: [T] = []  
    mutating func add(item: T) {  
        items.append(item)  
    }  
}
```

- The `T` placeholder is used wherever the associated type is used in the protocol, and when we create an instance of the `MyGenericType` type, we will need to define what type to use.

Advanced Generic Type Constraints

Generic Where Clause

- Generic where clauses allow us to add additional constraints to type parameters in generic functions and extensions. They provide more fine-grained control over type constraints, enabling us to express complex requirements.
- Example 2: Generic Where Clause

```
func allItemsMatch<C1: Container, C2: Container>(_ container1: C1, _ container2: C2) -> Bool  
where C1.Item == C2.Item, C1.Item: Equatable
```

- This function takes two containers and checks if all the items in the containers match each other. Let's break down the generic where clauses used in this function:

Advanced Generic Type Constraints

Generic Where Clause

- Example 2: Generic Where Clause

```
func allItemsMatch<C1: Container, C2: Container>(_ container1: C1, _ container2: C2) -> Bool
where C1.Item == C2.Item, C1.Item: Equatable
```

- This function takes two containers and checks if all the items in the containers match each other. Let's break down the generic where clauses used in this function:
- **C1: Container, C2: Container:** This part of the clause specifies that C1 and C2 must be types that conform to the Container protocol. It ensures that the function can only be called with containers that have the required capabilities (**append(_:), count, and subscript**) defined in the Container protocol.
- **C1.Item == C2.Item:** This clause states that the associated type Item of C1 must be the same as the associated type Item of C2. It ensures that both containers contain items of the same type.
- **C1.Item: Equatable:** This clause specifies that the associated type Item of C1 must conform to the Equatable protocol. It allows the function to compare the items for equality using the == operator.

Advanced Generic Type Constraints

Generic Where Clause

- With these generic where clauses, the function **`allItemsMatch(_:_:)`** can only be called with two containers (C1 and C2) that conform to the **`Container`** protocol, have the same associated type for `Item`, and have items that are `Equatable`. This provides compile-time safety and ensures that the function can only operate on compatible containers.

Advanced Generic Type Constraints

Class and Protocol Composition Constraints

- Swift allows us to apply more advanced constraints to generic types, including class and protocol composition constraints. These constraints allow us to specify that a generic type parameter must inherit from a specific class or conform to multiple protocols.
- Example

```
protocol Printable {  
    func printDescription()  
}  
  
class Vehicle {  
    var manufacturer: String  
  
    init(manufacturer: String) {  
        self.manufacturer = manufacturer  
    }  
}
```

```
class Car: Vehicle, Printable {  
    var model: String  
  
    init(manufacturer: String, model: String) {  
        self.model = model  
        super.init(manufacturer: manufacturer)  
    }  
  
    func printDescription() {  
        print("Car: \(manufacturer), \(model)")  
    }  
}  
  
func printVehicleDescription<T: Vehicle & Printable>(_ vehicle: T)  
{  
    vehicle.printDescription()  
}  
  
let myCar = Car(manufacturer: "Tesla", model: "Model 3")  
printVehicleDescription(myCar) // Output: Car: Tesla, Model 3
```

Advanced Generic Type Constraints

Class and Protocol Composition Constraints

- In this example, we define a protocol **Printable** and a class **Vehicle**. The class **Car** inherits from **Vehicle** and conforms to **Printable**. We then create a generic function **printVehicleDescription** that takes a type parameter **T** constrained to both **Vehicle** and **Printable**. This ensures that the function can only be called with types that satisfy both constraints.

```
protocol Printable {  
    func printDescription()  
}  
  
class Vehicle {  
    var manufacturer: String  
  
    init(manufacturer: String) {  
        self.manufacturer = manufacturer  
    }  
}
```

```
class Car: Vehicle, Printable {  
    var model: String  
  
    init(manufacturer: String, model: String) {  
        self.model = model  
        super.init(manufacturer: manufacturer)  
    }  
  
    func printDescription() {  
        print("Car: \(manufacturer), \(model)")  
    }  
}  
  
func printVehicleDescription<T: Vehicle & Printable>(_ vehicle: T)  
{  
    vehicle.printDescription()  
}  
  
let myCar = Car(manufacturer: "Tesla", model: "Model 3")  
printVehicleDescription(myCar) // Output: Car: Tesla, Model 3
```